

HandSDK Development Guidelines (Python Version)

Version History

Version	Modification Time	Modifier	Modify Content	Corresponding HandSDK version
v0.1	2026-03-16	Tan Zhiqiang	First Edition	v1.0.0
v0.2	2026-07-16	Zhang Zanyang	Modified the content and added explanations	v1.1.1

1. Overview of HandSDK

HandSDK is a unified dexterous hand control module that supports control interfaces for multiple types of dexterous hands, including the Gaia series dexterous hands and the Pantheon series dexterous hands, etc. It provides high-level abstraction interfaces and low-level control functions, and is applicable to robot research, education and industrial applications.

1.1 Key Features

- **Unified Interface:** Provides a standardized hand control interface that supports multiple hand types
- **Multi-hand Support:** Supports single-hand, dual-hand and blend mode
- **Real-time Control:** Supports real-time joint angle control and position control
- **Gesture Execution:** Multiple preset gestures are built in, and custom gestures are supported
- **Kinematic Calculation:** Provides forward and inverse kinematic calculation functions
- **Status Monitoring:** Real-time monitoring of motor status and hand status

1.2 Supported Dexterous Hands

1.2.1 Gaia Series Dexterous Hand

1.2.1.1 GaiaHand20

- **Communication Method:** CANFD communication
- **Control Accuracy:** High-precision position control
- **Degrees of Freedom:** 16 degrees of freedom (4 joints for the thumb + 3 joints for each of the other fingers)
- **Application Scenarios:** Fine manipulation, grasping tasks, gesture recognition

1.2.1.2 GaiaHand15

- **Communication Method:** CANFD communication
- **Control Accuracy:** High-precision position control
- **Degrees of Freedom:** 15 degrees of freedom (3 joints per finger $\times$ 5 fingers)
- **Application Scenarios:** Fine manipulation, grasping tasks, gesture recognition

1.2.2 Pantheon Series Dexterous Hand

1.2.2.1 PantheonHand20

- **Communication Method:** CAN bus communication
- **Control Accuracy:** High-precision position and torque control
- **Degrees of Freedom:** Configurable degrees of freedom (typically 15 degrees of freedom)
- **Application Scenarios:** Industrial applications, collaborative robots, research platforms

2. System Requirements

- **Supported systems:** Windows, Linux
- **Supported Languages:** Python 3.11+
- **Drive:**
 - PantheonHand20: CAN / CANFD Driver
 - Gaia series: Serial Port / CAN Driver



Given that most computers are equipped only with USB ports, the actual product utilizes a physical adapter to convert USB to CAN FD, CAN, or serial ports.

3. Environment Deployment

Create and activate an environment.

Code Block

```
1  conda create --name handsdk python=3.12
2
3  conda activate handsdk
```

4. HandSDK Installation

You only need to choose one of the following three methods for installation.

4.1 Offline Package Installation (wheel)

4.1.1 Download Link

1. [Cloud Drive Download](#)
2. Click the download button to get "02.HandSDK".

 01.HandGUI

 02.HandSDK

 03.URDF

4.1.2 Extract the folder

The decompressed folder is as follows

 docs

 example

 packages

4.1.3 Install the Hand module

If your system is Windows, select `handsdk-1.x.x-cpxxx-cpxxx-win_amd64.whl` for installation; if your system is Linux, select `handsdk-1.x.x-cpxxx-cpxxx-manylinux_xxx.whl` for installation. The installation steps are as follows:

Code Block

```
1  cd packages
2  pip install -r requirements.txt # Install requirements
3
4  # Install handsdk package on Windows
5  pip install handsdk-1.x.x-cpxxx-cpxxx-win_amd64.whl
6
7  # Install handsdk package on Linux(x86_64)
8  pip install handsdk-1.x.x-cpxxx-cpxxx-manylinux_2_35_x86_64.whl
9  # Install handsdk package on Linux(arm)
10 pip install handsdk-1.x.x-cpxxx-cpxxx-manylinux_2_38_aarch64.whl
```

4.2 Online Installation (PyPI)

Code Block

```
1  # Install directly
2  pip install handsdk
3
4  # Poor network conditions or needing a specific source
5  # Official source (recommended when mirrors are not synchronized)
6  pip install handsdk -i https://pypi.org/simple
7
8  # Tsinghua University mirror source
9  pip install handsdk -i https://pypi.tuna.tsinghua.edu.cn/simple
10
11 # Alibaba Cloud mirror source
12 pip install handsdk -i https://mirrors.aliyun.com/pypi/simple
13
14 # University of Science and Technology of China mirror source
15 pip install handsdk -i https://pypi.mirrors.ustc.edu.cn/simple
16
17 # Huawei Cloud mirror source
18 pip install handsdk -i https://repo.huaweicloud.com/repository/pypi/simple
19
20 # Southern University of Science and Technology mirror source
21 pip install handsdk -i https://mirrors.sustech.edu.cn/pypi/simple
```

5. Quick Start

Case 1: Basic Control of GaiaHand20

Code Block

```
1  import time
2  import math
3  from hand import create_hand
4  from hand.utils.serial_utils import auto_detect_gaia_ports
5
6  # Automatically detect serial ports
7  ports_config = auto_detect_gaia_ports()
8
9  # Create a right-hand GaiaHand20 instance.
10 hand = create_hand("gaiahand20", "right", port=ports_config['right'],
11                    use_slcan=True, has_main_board=True)
12
13 try:
14     if hand.connect():
15         print("GaiaHand20 successfully connected")
16
17         # Set motor smoothing level (optional)
18         # device_id=255 indicates broadcasting to all motors.
19         hand.config_pos_lpf_lv(device_id=255, level=3)
20         time.sleep(0.5)
21
22         # Enable all motors
23         hand.enable_all_motors_broadcast(True)
24         time.sleep(3)
25
26         # Control joint positions using list format (16 joints, in radians)
27         positions = [
28             # The four joints of the thumb
29             math.radians(20.0), 0.0, math.radians(27.0), math.radians(43.0),
30             # The three joints of the index finger
31             0.0, math.radians(43.0), math.radians(56.0),
32             # The three joints of the middle finger
33             0.0, math.radians(33.0), math.radians(25.0),
34             # The three joints of the ring finger
35             0.0, math.radians(28.0), math.radians(18.0),
36             # The three joints of the little finger
37             0.0, math.radians(19.0), math.radians(26.0)
38         ]
39
40         # Set the positions using broadcast mode
41         success = hand.move_joints_pos(positions, speed=1, use_broadcast=True)
```

```

41         print(f"Result of the positions setting: {'Success' if success else
'Failure'}")
42         time.sleep(2)
43
44         # Get current positions
45         current_positions = hand.get_joint_positions()
46         print(f"Current positions: {current_positions}")
47
48         # Hand go home
49         hand.hand_zero()
50         time.sleep(2)
51
52     finally:
53         if hand.is_connected():
54             # Disable all motors
55             hand.enable_all_motors_broadcast(False)
56             hand.close()

```

Case 2: GaiaHand20 Dual-Hand Mode

Code Block

```

1  import math
2  import time
3  from hand import create_hand
4  from hand.utils.serial_utils import auto_detect_gaia_ports
5
6  ports_config = auto_detect_gaia_ports()
7
8  # Create a dual-hand GaiaHand20 instance
9  hand = create_hand("gaiahand20", "double",
10                    left_port=ports_config['left'],
11                    right_port=ports_config['right'],
12                    baudrate=921600,
13                    use_slcan=True,
14                    has_main_board=True,
15                    joint_limit_enabled=False) # Disable limit checking (limit=
[0.000000, 1.570800], if it is set to True, the joint can execute even outside
this range, though a warning will be issued)
16
17  if hand.connect():
18      hand.enable_all_motors_broadcast(True)
19      time.sleep(2)
20
21      # Dual-hand mode: 32 joint positions (The first 16 values are for the right
hand, the last 16 values are for the left hand)

```

```

22     right_positions = [math.radians(5.0)] * 16 # Rotate all joints of the
    right hand by 5 degrees
23     left_positions = [math.radians(-5.0)] * 16 # Rotate all joints of the
    left hand by -5 degrees
24     double_positions = right_positions + left_positions
25
26     success = hand.move_joints_pos(double_positions, speed=0.5,
    use_broadcast=False)
27     print(f"Result of the dual-hand positions setting: {'success' if success
    else 'failure'}")
28     time.sleep(2)
29     hand.hand_zero()
30
31     hand.close()

```

Case 3: Smooth Transition Control of GaiaHand20

Code Block

```

1  import math
2  import time
3  from hand import create_hand
4  from hand.utils.serial_utils import auto_detect_gaia_ports
5  # Automatically detect serial ports
6  ports_config = auto_detect_gaia_ports()
7
8  hand = create_hand("gaiahand20", "right", port=ports_config['right'],
    baudrate=921600, use_slcan=True, has_main_board=True)
9
10 if hand.connect():
11     hand.enable_all_motors_broadcast(True)
12     time.sleep(2)
13
14     # Starting position (fully extended)
15     start_positions = [0.0] * 16
16
17     # End position (bent state)
18     end_positions = [
19         math.radians(30.0), 0.0, math.radians(27.0), math.radians(43.0), #
    Thumb
20         0.0, math.radians(43.0), math.radians(56.0), #
    Index finger
21         0.0, math.radians(33.0), math.radians(25.0), #
    Middle finger
22         0.0, math.radians(28.0), math.radians(18.0), #
    Ring finger

```

```

23         0.0, math.radians(19.0), math.radians(26.0) #
    Little finger
24     ]
25
26     # Smooth transition in 10 steps
27     for i in range(11):
28         t = i / 10.0 # interpolation coefficient
29         positions = []
30         for j in range(16):
31             pos = start_positions[j] + (end_positions[j] - start_positions[j])
    * t
32             positions.append(pos)
33
34         print(f"Step {i+1}/11: transition progress {t*100:.1f}%")
35         hand.move_joints_pos(positions, speed=0.5, use_broadcast=True)
36         time.sleep(0.1)
37
38     print("Smooth transition completed")
39     hand.hand_zero()
40     hand.close()

```

6. Use Cases

Check the cloud drive material `02.HandSDK\example` in 4.1.1

- GaiaHand20: refer to `gaiahand_example.py` ;
- PantheonHand20: refer to `pantheonhand_example.py` ;

7. Development Interface Specification

7.1 Create instance

7.1.1 create_hand ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Use the `create_hand ()` factory function to create hand instances

Function Signature:

Code block

```

1     # Factory function: create hand instances
2     def create_hand(

```



```

3     hand_type: Union[str, HandType],
4     hand_side: Union[str, HandSide] = HandSide.RIGHT,
5     **kwargs
6 ) -> Hand:

```

Parameter Description:

Parameter	Type	Description
hand_type	str or HandType	Hand type("pantheonhand20", "gaiahand20" or HandType enumeration)
hand_side	str or HandSide	Hand side ("right", "left", "double" or HandSide enumeration)
**kwargs	/	Other parameters, such as: - port , baudrate : serial port parameters (for Gaia series direct serial connection mode) - use_slcan : whether to use SLCAN CANFD communication (Gaia series, default: False) - slcan_channel : SLCAN serial port channel (e. g. 'COM6'), uses port if not specified - slcan_tty_baudrate : SLCAN serial port baud rate, default value is 115200 - slcan_arbitration_bitrate : SLCAN arbitration bitrate, default value is 1000000 - slcan_data_bitrate : SLCAN data segment baud rate, default value is 2000000 - can_config : CAN configuration

Return Value:

Parameter	Type	Description
Hand	class	Instances of hand control, actually returns a subclass of Hand

Usage Example:

Code Block

```

1  from hand import create_hand
2
3  # GaiaHand20 one-hand mode
4  hand = create_hand(
5      hand_type="gaiahand20",
6      hand_side="right",          # "right", "left", "double"
7      port="COM3",                # Serial port number
8      baudrate=921600,            # Baud rate (optional, default: 230400)
9      has_main_board=True         # With the main control board
10 )
11
12 # GaiaHand20 dual-hand mode
13 hand = create_hand(
14     hand_type="gaiahand20",
15     hand_side="double",
16     left_port="COM3",            # left-hand serial port
17     right_port="COM4"           # right-hand serial port
18 )
19
20 # PantheonHand20 one-hand mode
21 hand = create_hand(
22     hand_type="pantheonhand20",
23     hand_side="right",
24     can_config={                 # CAN configuration (optional)
25         'dev_type': 41,          # USB-CAN device type
26         'channel': 0,            # CAN channel
27         'arbitration_baudrate': 1000000, # Arbitration baud rate
28         'data_baudrate': 5000000, # Data phase baud rate
29         'canfd_mode': 0          # CANFD mode
30     }
31 )
32
33 # PantheonHand20 dual-hand mode
34 hand = create_hand(
35     hand_type="pantheonhand20",
36     hand_side="double"
37 )

```

7.1.2 auto_detect_gaia_ports()

Applicable Objects: GaiaHand20, GaiaHand15

Feature Description: Automatically detect available serial ports

Function Signature:

Code block

```
1 def auto_detect_gaia_ports() -> Optional[Dict[str, str]]:
```

Return Value:

Parameter	Type	Description
/	Dict[str, str] or None	Returns the detected available serial ports, None indicates that no available serial port is detected

Usage Example:

Code Block

```
1 from hand.utils.serial_utils import auto_detect_gaia_ports
2
3 # Automatically detect available serial ports
4 ports_config = auto_detect_gaia_ports()
5
6 if ports_config and ports_config['available']:
7     print(f"left-hand serial port: {ports_config['left']}")
8     print(f"right-hand serial port: {ports_config['right']}")
9
10 # Create an instance using the detected serial port.
11 hand = create_hand("gaiahand20", "right", port=ports_config['right'])
```

7.2 Connection Management

7.2.1 connect ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Connect hand hardware and update connection status

Function Signature:

Code block

```
1 def connect(self) -> bool:
```

Return Value:

Parameter	Type	Description
True	bool	Connection successful
False	bool	Connection failed

Usage Example:

Code Block

```
1  if hand.connect():
2      print("Connection successful")
3  else:
4      print("Connection failed")
```

7.2.2 disconnect ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Disconnect the connection

Function Signature:

Code block

```
1  def disconnect(self):
```

Return Value:

Parameter	Type	Description
None	None	Performs the operation: disconnect the currently created hand instance; no specific return value

Usage Example:

Code Block

```
1  hand.disconnect()
```

7.2.3 is_connected ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Check connection status

Function Signature:

Code block

```
1 def is_connected(self) -> bool:
```

Return Value:

Parameter	Type	Description
True	bool	Connected
False	bool	Not connected

Usage Example:

Code Block

```
1 if hand.is_connected():
2     print("Connected successfully")
```

7.2.4 close ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Close the connection (recommended)

Function Signature:

Code block

```
1 def close(self):
```

Return Value:

Parameter	Type	Description
None	None	Perform the operation: disconnect and clean up resources; no specific return value

Usage Example:

Code Block

```
1  # Automatically disconnect and clean up resources
2  hand.close()
```

7.2.5 Use a context manager

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Automatically manage resource entry and exit(recommended)

Usage Example:

Code Block

```
1  with create_hand("gaiahand20", "right", port="COM3") as hand:
2      if hand.connect():
3          # Perform operation
4          hand.enable_all_motors(True)
5          # ...
6      # Automatically close the connection
```

7.3 Motor enable control

7.3.1 enable_motor ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Enable/ disable a single motor

Function Signature:

Code block

```
1  def enable_motor(
2      self,
3      motor_id: int,
4      enable: bool = True
5  ) -> bool:
```

Parameter Description:

Parameter	Type	Description
<code>motor_id</code>	<code>int</code>	Motor ID

<code>enable</code>	<code>bool</code>	Enable or not
---------------------	-------------------	---------------

Return Value:

Parameter	Type	Description
<code>True</code>	<code>bool</code>	Operation successful
<code>False</code>	<code>bool</code>	Operation failed

Usage Example:

Code Block

```
1  # Enable the motor with ID 1
2  hand.enable_motor(1, True)
3
4  # Disable the motor with ID 1
5  hand.enable_motor(1, False)
```

7.3.2 get_motor_id()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Function Description: Get Motor ID

Function Signature:

Code block

```
1  def get_motor_id(
2      self,
3      finger: FingerType,
4      joint: JointType
5  ) -> int:
```

Parameter Description:

Parameter	Type	Description
<code>finger</code>	<code>FingerType</code>	Finger Type
<code>joint</code>	<code>JointType</code>	Joint Type

Return Value:

Parameter	Type	Description
<code>/</code>	<code>int</code>	Motor ID

Usage Example:

Code block

```
1  from hand import FingerType, JointType
2
3  motor_id = hand.get_motor_id(FingerType.THUMB,JointType.JOINT_1)
4  print(f"motor id : {motor_id}")
```

7.3.3 enable_all_motors ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Enable/ disable all motors

Function Signature:

Code block

```
1  def enable_all_motors(
2      self,
3      enable: bool = True
4  ) -> bool:
```

Parameter Description:

Parameter	Type	Description
<code>enable</code>	<code>bool</code>	Enable or not

Return Value:

Parameter	Type	Description
<code>True</code>	<code>bool</code>	Operation successful
<code>False</code>	<code>bool</code>	Operation failed

Usage Example:

Code Block

```
1  # Enable all motors
2  hand.enable_all_motors(True)
3
4  # Disable all motors
5  hand.enable_all_motors(False)
```

7.3.4 enable_all_motors_broadcast ()

Applicable Objects: GaiaHand20, GaiaHand15

Feature Description: Broadcast to enable/ disable all motors

Function Signature:

Code block

```
1  def enable_motors_broadcast(
2      self,
3      enable_states: Dict[int, bool]
4  ) -> bool:
```

Parameter Description:

Parameter	Type	Description
enable_states	Dict[int, bool]	Mapping of motor IDs to enable states

Return Value:

Parameter	Type	Description
True	bool	Operation successful
False	bool	Operation failed

Usage Example:

Code Block

```

1  # Broadcast-enable all motors (more efficient)
2  hand.enable_all_motors_broadcast(True)
3
4  # Broadcast-disable all motors
5  hand.enable_all_motors_broadcast(False)

```

7.4 Joint position control

7.4.1 move_joints_pos ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Batch set joint positions, this is the most commonly used joint control method, which supports two formats: list and dictionary.

Function Signature:

Code Block

```

1  def move_joints_pos(
2      self,
3      positions: Union[List[float], Dict],
4      speed: float = 0.5,
5      use_broadcast: bool = False
6  ) -> bool

```

Parameter Description:

Parameter	Type	Description
<code>positions</code>	<code>List[float]</code> or <code>Dict</code>	Joint position data; supports list format and dictionary format
<code>speed</code>	<code>float</code>	Motion speed ratio (0-1)
<code>use_broadcast</code>	<code>bool</code>	Whether to use broadcast mode; True uses <code>set_joint_angles_broadcast</code> to set all joints via broadcast, while False uses <code>set_joint_angle</code> to set them one by one

 PantheonHand20 does not support broadcast mode, the `use_broadcast` parameter will be ignored

- **positions:** Joint position data, in radians
 - **List format:**
 - **GaiaHand20:** contains **16** (single-hand) or **32** (dual-hand) position data points
 - One-hand mode sequence: **Thumb (4 joints)** + Index finger (3 joints) + Middle finger (3 joints) + Ring finger (3 joints) + Little finger (3 joints)
 - Dual-hand mode sequence: Right hand (16 joints) + Left hand (16 joints)
 - **PantheonHand20:** Contains 15 position data points (for single hand) or 30 position data points (for both hands)
 - One-hand mode sequence: Thumb (3 joints) + Index finger (3 joints) + Middle finger (3 joints) + Ring finger (3 joints) + Little finger (3 joints)
 - Dual-hand mode sequence: Right hand (15 joints) + Left hand (15 joints)
 - **Dictionary format:** more intuitive Structured Data

Code Block

```

1  # Taking GaiaHand20 as an example
2  # List format: Contains 16 (one-hand) or 32 (dual-hand) position data (in
   # radians)
3  [
4      angle_1, angle_2, angle_3, angle_4,           # Thumb
5      angle_1, angle_2, angle_3,                   # Index finger
6      ...                                           # In order: middle finger,
   # ring finger, little finger
7  ]
8
9  # Dictionary format: more intuitive structured data
10 {
11     1: {                                           # Right hand (1=right hand,
   # 2=left hand)
12         1: [angle_1, angle_2, angle_3, angle_4], # Thumb
13         2: [angle_1, angle_2, angle_3],          # Index finger
14         ...                                       # 1=thumb, 2=index finger,
   # 3=middle finger, 4=ring finger, 5=little finger
15     },
16     2: {                                           # left hand (optional, using
   # in dual-hand mode)
17         ...
18     }
19 }
```

Return Value:

Parameter	Type	Description
True	bool	Operation successful
False	bool	Operation failed

Usage Example:

Code Block

```
1  import math
2  # Taking GaiaHand20 as an example
3  # List format (16 joints)
4  positions = [
5      math.radians(30.0), 0.0, math.radians(27.0), math.radians(43.0), # Thumb
6      0.0, math.radians(43.0), math.radians(56.0), # Index
7      0.0, math.radians(33.0), math.radians(25.0), # Middle
8      0.0, math.radians(28.0), math.radians(18.0), # Ring
9      0.0, math.radians(19.0), math.radians(26.0) # Little
10 ]
11 hand.move_joints_pos(positions, speed=1, use_broadcast=True)
12
13 # Dictionary format
14 positions_dict = {
15     1: {
16         1: [math.radians(30.0), 0.0, math.radians(10.0), math.radians(10.0)],
17         2: [0.0, math.radians(10.0), math.radians(10.0)],
18         3: [0.0, math.radians(13.0), math.radians(15.0)],
19         4: [0.0, math.radians(18.0), math.radians(10.0)],
20         5: [0.0, math.radians(19.0), math.radians(10.0)]
21     }
22 }
23 hand.move_joints_pos(positions_dict, speed=1, use_broadcast=True)
```

7.4.2 get_joints_pos_vel ()

Applicable Objects: GaiaHand20, GaiaHand15

Feature Description: Get the joint position (rad) and angular velocity (rad/s)

Function Signature:

Code Block

```
1  def get_joints_pos_vel(  
2      self,  
3      finger: Optional[FingerType] = None,  
4      joint: Optional[JointType] = None,  
5      timeout: float = 0.2,  
6  ) -> Dict[int, Dict[int, List[Optional[float]]]]:
```

Parameter Description:

Parameter	Type	Description
finger	FingerType or None	Finger; when omitted together with joint , it refers to the entire hand (the side covered by the current adapter or both hands).
joint	JointType or None	Joint; when used together with finger and both are omitted at the same time, it refers to the entire hand
timeout	float	Single-axis/Batch Read Timeout (seconds)

Return Value:

Output the corresponding input finger joint positions (rad) and angular velocities (rad/s) in dictionary format, protocol alignment move_joints_pos Output dictionary:

Code Block

```
1  {  
2      3: {                                     # Current position of the right  
        hand (3,5=right hand; 4,6=left hand)  
3          1: [pos_1, pos_2, pos_3, pos_4],    # Current thumb position  
4          2: [pos_1, pos_2, pos_3],          # Current position of the index  
        finger  
5          ...                                # 1=thumb, 2=index finger, 3=middle  
        finger, 4=ring finger, 5=little finger  
6      },
```

```

7         4: {                                     # Current position of the left hand
8             1: [...],
9             2: [...],
10            ...
11        },
12        5: {                                     # Current velocity of the right
            hand
13            1: [vel_1, vel_2, vel_3, vel_4],      # Current thumb velocity
14            2: [vel_1, vel_2, vel_3],            # Current velocity of the index
            finger
15            ...                                   # 1=thumb, 2=index finger, 3=middle
            finger, 4=ring finger, 5=little finger
16        },
17        6: {                                     # Current velocity of the left
            hand
18            1: [...],
19            2: [...],
20            ...
21        }
22    }

```

Usage Example:

Code Block

```

1  from hand import FingerType, JointType
2
3  # Get the joint positions and angular velocities for the entire hand by default
4  fullhand_pos_vel = hand.get_joints_pos_vel()
5
6  # Get the specified joint positions and angular velocities
7  joint_pos_vel = hand.get_joints_pos_vel(FingerType.THUMB, JointType.JOINT_1,
    timeout=0.2)

```

7.4.3 get_joint_positions ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Get the joint position (rad)

Function Signature:

Code Block

```

1  # GaiaHand20
2  def get_joint_positions(
3      self,

```

```

4     joint_names: Optional[List[str]] = None,
5     sync: bool = False,
6     timeout: Optional[float] = None
7 ) -> Union[np.ndarray, List[float]]:
8
9     # PantheonHand20
10    def get_joint_positions(
11        self,
12        joint_names: Optional[List[str]] = None
13    ) -> np.ndarray:

```

Parameter Description:

Parameter	Type	Description
<code>joint_names</code>	<code>List[str]</code> or <code>None</code>	List of joint names; returns all joints when set to <code>None</code>
<code>sync</code>	<code>bool</code>	<code>True</code> = synchronous mode, <code>False</code> = asynchronous mode
<code>timeout</code>	<code>float</code> or <code>None</code>	Synchronous wait timeout (seconds)

Return Value:

Hand Type	Parameter	Type	Description
GaiaHand20	<code>angles_list</code>	<code>np.ndarray</code> or <code>List[float]</code>	Joint position array (in radians)
PantheonHand 20	<code>/</code>	<code>np.ndarray</code>	Joint position array (in radians)

Usage Example:

Code Block

```

1     # Get all joint positions (asynchronous mode, recommended)
2     positions = hand.get_joint_positions(sync=False)
3
4     # Get all joint positions (synchronous mode)
5     positions = hand.get_joint_positions(sync=True, timeout=0.1)
6
7     # Return format:

```

```
8  # - GaiaHand20: list, containing 16(one-hand) or 32(dual-hand) positions(in
    radians)
9  # - PantheonHand20: ditionary, and the key is the finger ID, the value is the
    list of joint angles for that finger
```

7.4.4 set_joint_angle ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Set the angle of a single joint

Function Signature:

Code Block

```
1  def set_joint_angle(
2      self,
3      finger: FingerType,
4      joint: JointType,
5      angle: float,
6      speed: float = 0.85
7  ) -> bool:
```

Parameter Description:

Parameter	Type	Description
<code>finger</code>	<code>FingerType</code>	Finger type
<code>joint</code>	<code>JointType</code>	Joint type
<code>angle</code>	<code>float</code>	Target angle (radians)
<code>speed</code>	<code>float</code>	Motion speed ratio

Return Value:

Parameter	Type	Description
<code>True</code>	<code>bool</code>	Operation successful
<code>False</code>	<code>bool</code>	Operation failed

Usage Example:


```

1 Code Block import FingerType, JointType
2
3 # Set the angle of the index finger's first joint to 45 degrees (in radians)
4 hand.set_joint_angle(
5     finger=FingerType.INDEX,
6     joint=JointType.JOINT_1,
7     angle=math.radians(45),
8     speed=0.5
9 )
10
11 # Return:
12 # True=succeeded to set; False=failed to set

```

7.4.5 get_joint_angle ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Get the angle of a single joint

Function Signature:

Code Block

```

1 def get_joint_angle(
2     self,
3     finger: FingerType,
4     joint: JointType
5 ) -> float:

```

Parameter Description:

Parameter	Type	Description
finger	FingerType	Finger type
joint	JointType	Joint type

Return Value:

Parameter	Type	Description
/	float	Single joint angle (in radians)

Usage Example:

Code Block

```
1  from hand import FingerType, JointType
2
3  # Get the angle of the index finger's first joint.
4  angle = hand.get_joint_angle(
5      finger=FingerType.INDEX,
6      joint=JointType.JOINT_1
7  )
8  print(f"angle: {math.degrees(angle):.2f}degrees")
```

7.5 Gesture Control

7.5.1 hand_zero ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Hand go home

Function Signature:

Code block

```
1  def hand_zero(self) -> bool:
```

Return Value:

Parameters	Type	Description
True	bool	Operation successful
False	bool	Operation failed

Usage example:

Code Block

```
1  # Perform the homing operation to return all joints to their initial positions.
2  success = hand.hand_zero()
3  if success:
4      print("Homing successfully")
```

7.6 Motor Status Query

7.6.1 get_motor_status ()

Applicable Objects: GaiaHand20, GaiaHand15, PantheonHand20

Feature Description: Get motor status

Function Signature:

Code block

```
1  # GaiaHand20
2  def get_motor_status(
3      self,
4      motor_id: Optional[int] = None,
5      sync: bool = False,
6      timeout: float = 1.0
7  ) -> Dict:
8
9  # PantheonHand20
10 def get_motor_status(
11     self,
12     motor_id: Optional[int] = None
13 ) -> Dict:
```

Parameter Description:

Parameters	Type	Description
<code>motor_id</code>	<code>int</code> or <code>None</code>	Specify the motor ID; when it is <code>None</code> , all motors will be read
<code>sync</code>	<code>bool</code>	<code>True</code> = synchronous mode, <code>False</code> = asynchronous mode
<code>timeout</code>	<code>float</code>	Synchronous Wait Timeout (seconds)

Return Value:

Parameters	Type	Description
<code>status</code>	<code>Dict</code>	Motor status dictionary; the two-hand mode returns data aggregated by hand side

Usage Example:

Code Block

```
1  # Get all motor statuses
2  all_status = hand.get_motor_status()
3
4  # Get the status of the specified motor
5  motor_status = hand.get_motor_status(motor_id=0)
6  print(f"motor status: {motor_status}")
```

7.7 Motor Smoothness Level Setting

7.7.1 config_pos_lpf_lv ()

Applicable Objects: GaiaHand20, GaiaHand15

Feature Description: Set position smoothness level

Function Signature:

Code block

```
1  def config_pos_lpf_lv(
2      self,
3      device_id: Optional[int] = None,
4      level: int = None,
5      callback: Optional[callable] = None
6  ) -> str:
```

Parameter Description:

Parameters	Type	Description
device_id	int or None	Device ID, None indicates the use of the default value, and 255 indicates broadcasting to all motors
level	int	Smoothing Level (0-5)
callback	callable or None	Callback function (optional)

Return Value:

--	--	--

Parameters	Type	Description
/	str	Motor status dictionary; the two-hand mode returns data aggregated by hand side

Usage Example:

Code Block

```

1  # Set the smoothing level for all motors (device_id=255 indicates broadcasting
   # to all motors).
2  # Level range: 0-5, higher values yield better smoothing results.
3  hand.config_pos_lpf_lv(device_id=255, level=3)
4
5  # Set the smoothing level for single motors
6  hand.config_pos_lpf_lv(device_id=0, level=3)
```

7.8 Enumeration Type

7.8.1 HandType

Type Description: Hand type

Code Block

```

1  from hand import HandType
2
3  HandType.GAIA_HAND_15      # GaiaHand15 (15 joints)
4  HandType.GAIA_HAND_20      # GaiaHand20 (16 joints)
5  HandType.PANTHEON_HAND_20  # PantheonHand20
```

7.8.2 HandSide

Type Description: Lateral side of the Hand

Code Block

```

1  from hand import HandSide
2
3  HandSide.RIGHT    # Right hand
4  HandSide.LEFT     # Left hand
5  HandSide.DOUBLE   # Double hand
```

7.8.3 FingerType

Type Description: Finger type

Code Block

```
1  from hand import FingerType
2
3  FingerType.THUMB    # Thumb
4  FingerType.INDEX    # Index finger
5  FingerType.MIDDLE   # Middle finger
6  FingerType.RING     # Ring finger
7  FingerType.LITTLE   # Little finger
```

7.8.4 JointType

Type Description: Joint type

Code Block

```
1  from hand import JointType
2
3  JointType.JOINT_1   # Joint 1(MCP_1, MCP Abduction-Adduction)
4  JointType.JOINT_2   # Joint 2(MCP_2, MCP Flexion/Extension)
5  JointType.JOINT_3   # Joint 3(PIP)
6  JointType.JOINT_4   # Joint 4(DIP, only thumb in GaiaHand20)
```

JointType	Anatomic al Definitio n	DOF	Positive / Negative Direction
JOINT_1	MCP_1	Abduction / Adduction	Movement of the fingers away from the middle finger (abduction) is positive; movement toward the middle finger (adduction) is negative.
JOINT_2	MCP_2	Flexion/Exten sion	Curving the fingers toward the palm (flexion) is positive; straightening the fingers(extension) is negative.
JOINT_3	PIP	Flexion/Exten sion	Curving the fingers toward the palm (flexion) is positive; straightening the fingers(extension) is negative.
JOINT_4 (only thumb in	DIP	Flexion/Exten sion	Curving the fingers toward the palm (flexion) is positive; straightening the fingers(extension) is negative.

7.9 Frequently Asked Questions

Serial No.	Issue	Common Causes	Solution
1	Connection failed	Invalid serial port number, hardware not connected, driver not installed	Check whether the serial port number is correct, confirm the hardware connection, install the serial port driver, and use <code>auto_detect_gaia_ports ()</code> for automatic detection
2	Communication timeout	Baud rate mismatch and poor communication line quality	Check the baud rate setting (Gaia series defaults to 230400 or 921600), verify the USB cable connection, and try again with a lower baud rate
3	Motor does not respond	Motor not enabled, power not connected	Call <code>enable_all_motors (True)</code> to enable the motors, check the power connection, and wait for the enable to stabilize (2 seconds is recommended)
4	Inaccurate position control	Not homed, encoder abnormality, angle unit error	Execute the <code>hand_zero ()</code> homing operation, check the encoder status, confirm the angular unit (radian system), and verify the kinematic parameters
5	Unstable broadcast mode	communication interference and inconsistent motor response	Use non-broadcast mode (set <code>use_broadcast</code> to <code>False</code>), lower the speed parameter and increase the transmission interval
6	Dual-hand mode connection failed	Serial port configuration error, hardware mismatch	Confirm that the left and right hand serial ports are correct, check whether the hardware of both hands is connected, and verify the serial port permissions (for Linux systems)
7	CAN device connection failed (PantheonHand20)	The CAN driver is not installed, or the device has insufficient permissions	Install the CAN driver (ZLG CAN driver), configure the udev rules for the Linux system, and check the device permissions
8	Abnormal angle value	Unit confusion (degrees/radians), out	Use <code>math.radians ()</code> to convert degrees to radians, check the angle range (typically

		of range	-90° to 90°), and review the error log
9	The smooth transition is not smooth	Insufficient steps and excessively high speed	Increase the interpolation steps (10-20 steps are recommended), lower the speed parameter, and increase the delay time per step
10	Get position returns None	Asynchronous Mode: no waiting, communication timeout	Use Synchronization Mode (sync= True), increase the timeout duration, and check the connection status

7.10 Debugging Tips

1. Enable verbose logging:

Code Block

```
1 from hand.core import enable_all_logs, set_log_level
2
3 enable_all_logs()
4 set_log_level('DEBUG')
```

2. Check serial port connection:

Code Block

```
1 from hand.utils.serial_utils import get_available_ports
2
3 ports = get_available_ports()
4 print(f"Serial ports available: {ports}")
```

3. Verify connection status:

Code Block

```
1 if hand.is_connected():
2     print("Connected")
3     status = hand.get_motor_status()
4     print(f"Motor status: {status}")
5 else:
6     print("Not connected")
```

4. Test basic functions:


```

1 # Block the connection first
2 if hand.connect():
3     print("Connected successfully")
4
5     # Test enable
6     if hand.enable_all_motors(True):
7         print("Enabled successfully")
8         time.sleep(2)
9
10    # Test homing
11    if hand.hand_zero():
12        print("Homing successfully")

```

7.11 Notes

1. **Angle Unit:** All angle parameters use **radian measure**, and you can use `math.radians ()` to convert degrees to radians
2. **Enable Waiting:** It is recommended to wait for 1-2 seconds after enabling the motor to ensure that the motor initialization is completed
3. **Connection Sequence:** Connect the hardware first, then enable the motor, and finally execute the control command
4. **Disconnection Sequence:** First disable the motor, then disconnect the connection, which will be automatically handled via the `close ()` method
5. **Broadcast Mode:** The broadcast mode of Gaia series features faster speed yet may be less stable, so it is recommended to prioritize using non-broadcast modes.
6. **Smoothing Level:** Gaia series supports configuring the smoothing level (0-5); a higher value delivers better smoothing effect, yet may result in slower response.
7. **Dual-Hand Mode:** In dual-hand mode, the sequence of position data is: right hand (15 joints) + left hand (15 joints)
8. **Error Handling:** It is recommended to use try-except blocks to catch exceptions and ensure proper resource release
9. **Thread Safety:** Pay attention to synchronization in a multi-threaded environment to avoid concurrent control over the same hand component
10. **Linux Permissions:** Under Linux systems, you may need to configure udev rules or use sudo privileges

8. Common Problems and Solutions

8.1 Possible Solutions for the Undetectable ttyUSB Interface by GUI on Linux Systems

8.1.1 Problem Description:

You have connected GaiaHand20 or PantheonHand20 to the computer's serial port, but no connectable ttyUSB serial port appears.

8.1.2 Possible Causes:

The USB serial port is occupied by brltty (the text accessibility tool for visually impaired users preinstalled in Ubuntu), making it impossible to connect to the driver control board.

8.1.3 Troubleshooting Methods:

First, enter the following command via the terminal:

Code Block

```
1  ls /dev/tty*
```

List the serial ports to confirm that there is no ttyUSB serial port input

Then, enter the following command to call up the system log:

Code Block

```
1  sudo dmesg | tail -20
```

Therefore, you should be able to get an output similar to the following log

Code Block

```
1  [ 361.978901] input: BRLTTY 6.4 Linux Screen Driver Keyboard as
    /devices/virtual/input/input9
2  [ 362.137630] usb 1-2: usbfs: interface 0 claimed by ch341 while 'brltty'
    sets config #1
3  [ 362.220296] ch341-uart ttyUSB0: ch341-uart converter now disconnected from
    ttyUSB0
4  [ 362.220296] ch341 1-2:1.0: device disconnected
5  [ 450.348603] usb 1-2: USB disconnect, device number 3
6  [ 494.264072] usb 1-2: new full-speed USB device number 4 using ohci-pci
7  [ 494.743818] usb 1-2: New USB device found, idVendor=1a86, idProduct=7523,
    bcdDevice= 2.64
8  [ 494.743818] usb 1-2: New USB device strings: Mfr=0, Product=2,
    SerialNumber=0
```

```
9 [ 494.743818] usb 1-2: Product: USB Serial
10 [ 494.747684] ch341 1-2:1.0: ch341-uart converter detected
11 [ 494.780224] usb 1-2: ch341-uart converter now attached to ttyUSB0
12 [ 495.387325] input: BRLTTY 6.4 Linux Screen Driver Keyboard as
    /devices/virtual/input/input10
13 [ 495.427016] usb 1-2: usbfs: interface 0 claimed by ch341 while 'brltty'
    sets config #1
14 [ 495.465354] ch341-uart ttyUSB0: ch341-uart converter now disconnected from
    ttyUSB0
15 [ 495.465354] ch341 1-2:1.0: device disconnected
```

If a statement similar to the following appears, it can be confirmed that brltty is occupying the resource:

Code Block

```
1 [ 362.137630] usb 1-2: usbfs: interface 0 claimed by ch341 while 'brltty'
    sets config #1
2 [ 362.220296] ch341-uart ttyUSB0: ch341-uart converter now disconnected from
    ttyUSB0
```

8.1.4 Solution

8.1.4.1 Disable brltty

Enter the following command in the terminal:

Code Block

```
1 tellarobot@stellarobot:~$ sudo systemctl stop brltty
2 stellarobot@stellarobot:~$ sudo systemctl disable brltty
3 stellarobot@stellarobot:~$ systemctl status brltty
```

If any of the following occurs

Code Block

```
1 rltty.service - Braille Device Support
2     Loaded: loaded (/lib/systemd/system/brltty.service; disabled; vendor
    preset: enabled)
3     Active: inactive (dead)
4     Docs: man:brltty(1)
5           http://brltty.com/
```

this indicates that brltty has been disabled. In this case, please reconnect the USB device and access the tty serial port via the corresponding command

Code Block

```
1  ls /dev/tty*
```

Check whether ttyUSB appears. If it does, it means the USB serial port is available.

8.1.4.2 Remove brltty (proceed with caution)

If the serial port still cannot be detected after disabling brltty, you can delete brltty with the following command:

Code Block

```
1  sudo apt remove --purge brltty
2  sudo apt autoremove
```

After that, please reconnect the device and access the tty serial port. If the ttyUSB serial port appears, the GUI can be used normally.

8.2 Permission Denied for /dev/ttyACM0 on Linux System

8.2.1 Problem Description

When connecting to GaiaHand20 using HandSDK, the following connection failure message may appear:

Code Block

```
1  SLCAN fail to connect: Failed to connect: open failed: /dev/ttyACM0
```

8.2.2 Solution

Enter the following command in the terminal:

Code Block

```
1  sudo chmod 777 /dev/ttyACM0
```

8.3 The Slcan Dynamic Library is Missing on Linux Systems

8.3.1 Problem Description

When connecting to GaiaHand20 using HandSDK, the following message appears:

Code Block

```
1  /lib/x86_64-linux-gnu/libc.so.6: version `GLIBC_2.34' not found (required by  
   /home/guest/handgui/src/handsdk/hand/can_utils/slcan/libslcan_core_linux_x86_64  
   .so)
```

8.3.2 Solution

Contact the technical after-sales personnel